Entregable 3
Resiliencia y Observabilidad

UNIVERSIDAD EAFIT | Inspira Crea Transforma

ENTREGABLE 3

El corazón del proyecto

Fallas configurables · Experimentación · Observabilidad

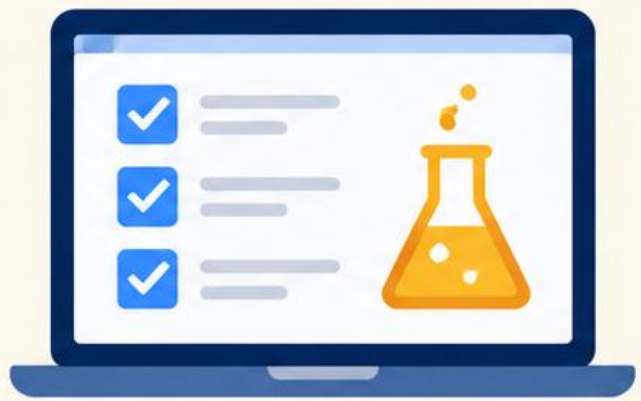
1 Configura las fallas

4 familias, 1–2 fallas por familia.

- Latencia
- Base de datos
- Degradación
- Disponibilidad

2 Experimenta

Ejecuta 4–8 experimentos (1–2 fallas por familia).



3 Observa y explica

Analiza qué pasa, cómo detectarlo, por qué ocurre y qué decisión tomar.



PRINCIPIO

La falla no es el resultado.
El resultado es lo que aprendemos al observar la respuesta del sistema.

UNIVERSIDAD EAFIT | Inspira Crea Transforma

CONDICIÓN FUNDAMENTAL

¿Qué significa “falla configurable”?

Toda falla debe cumplir cinco condiciones.



Sin redespiegue

Archivo, variable o endpoint administrativo.



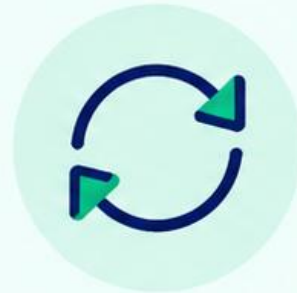
Parametrizada

Intensidad, probabilidad, alcance, duración.



Focalizable

Endpoint, módulo o % del tráfico cuando aplique.



Reversible

Kill switch durante la demostración.



Reproducible

Misma configuración + semilla → comportamiento comparable.



Una falla que solo lanza una excepción no cumple la definición de falla configurable.

UNIVERSIDAD EAFIT | Inspira Crea Transforma

CATÁLOGO

Cuatro familias · 1–2 fallas por familia

No se compensa la ausencia de una familia con más fallas de otra.

A Latencia inducida

- Latencia fija
- Jitter
- Pérdida/timeout
- Latencia parcial



B Falla de base de datos

- Caída temporal
- Pool agotado
- Bloqueo
- Failover



C Degradación de tiempos

- CPU
- Memoria/GC
- Pool de hilos
- Consumidor pausado



D Riesgo de disponibilidad

- Instancia caída
- Partición de red
- DNS
- 5xx externos
- Reloj





El objetivo es experimentar, observar y aprender.

Una falla que solo lanza una excepción no cumple la definición.

UNIVERSIDAD EAFIT | Inspira Crea Transforma

OBSERVABILIDAD

Descubrir qué señales permiten entender la falla

No se evalúa tener muchas herramientas; se evalúa su utilidad.

Métricas



Rendimiento y recursos del sistema.

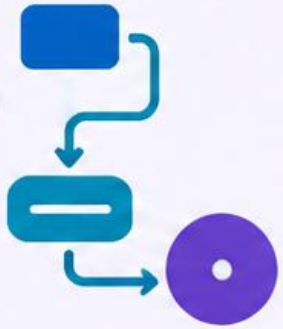
- p50/p95/p99
- CPU
- Memoria
- GC / colas

Logs



Qué ocurrió y en qué componente.

Trazas



Cómo se propaga la solicitud en componentes.



EL EQUIPO DEBE PODER RESPONDER

1. ¿Qué ocurrió?
2. ¿Dónde está la falla?
3. ¿Qué la está causando?
4. ¿Qué señales fueron útiles?
5. ¿Cuáles no lo fueron?

UNIVERSIDAD EAFIT | Inspira Crea Transforma

APRENDIZAJE

¿Qué deben descubrir sobre observabilidad?



Ventajas

Detectar degradación, localizar problemas y comparar antes/después.



Desventajas

Costo, ruido, overhead y falsos positivos.



Usos posibles

Explicar incidentes, mejorar SLO/SLA y orientar decisiones.

UNIVERSIDAD EAFIT | Inspira Crea Transforma

RÚBRICA

Evaluación · 100 puntos

La observabilidad y el método experimental concentran el mayor peso.

Fallas configurables	20%
Diseño experimental y predicciones	20%
Ejecución, reproducibilidad y reversibilidad	15%
Análisis de resultados y aprendizaje	10%
Observabilidad y evidencia	25%
Corrección, reexperimentación y comunicación	10%

Observabilidad y evidencia = 25%

Criterio de mayor peso

¿Qué debe quedar en el repositorio?



Configuración

Catálogo, parámetros, targets y switches.



Código

Base → falla → medición → corrección → repetición.



Evidencia

Experimentos, resultados, logs y métricas.



Informe

Máximo 10 páginas + anexos técnicos.



Tabla obligatoria:

ID · familia · falla · configuración · predicción · resultado · evidencia · aprendizaje

Condiciones mínimas

Además de la nota de la rúbrica, todas deben cumplirse.



4–8 experimentos
1–2 por familia.



Cuatro familias
con fallas configurables.



Activación sin redepliegue.



Predicción previa documentada.



Procedimiento reproducible.



Kill switch funcional.



Evidencia observable
por experimento.



Análisis y aprendizaje
de cada experimento.

Una demostración de falla no es un experimento



Gráficas sin hipótesis ni contexto.



“La aplicación se cayó” como único resultado.



Muchos dashboards sin interpretación.



Promedios como única medición de latencia.



La diferencia entre la predicción y el resultado es la nota.

UNIVERSIDAD EAFIT

ENTREGABLE 3

El corazón del proyecto

Fallas configurables, experimentación y observabilidad

Aplicaciones y Sistemas Escalables

Entender. Crear. Transformar.

1. Objetivo

Construir un laboratorio de fallas configurable y reproducible sobre el sistema desarrollado, para entender qué ocurre cuando falla, cómo detectarlo con observabilidad y qué decisiones de diseño tomar.

2. Fallas configurables

Cada falla debe cumplir las cinco condiciones:

1. **Sin redespliegue** — Se activa por archivo, variable o endpoint admin.
2. **Parametrizada** — Intensidad, probabilidad, alcance, duración.
3. **Focalizable** — Endpoint, componente o % de tráfico.
4. **Reversible en segundos** — Kill switch durante la demostración.
5. **Reproducible** — Misma config. + misma semilla = mismo comportamiento.

3. Cuatro familias de fallas (mínimo dos por familia)

A · Latencia inducida
- Latencia fija en dependencia
- Jitter (distribución lognormal)
- Solo en parte del tráfico
- Timeout controlado

B · Falla de base de datos
- Caída temporal de PostgreSQL
- Pool de conexiones agotado
- Bloqueo sobre un recurso
- Failover o disco lleno simulado

C · Degradación de tiempos
- Saturación de CPU
- Presión de memoria y GC
- Inanición del pool de hilos
- Consumidor pausado / lag

D · Riesgo de disponibilidad
- Caída de una instancia
- Partición de red servicio ↔ base
- DNS que no resuelve
- Respuestas 5xx externas
- Desfase de reloj

4. Método experimental

Cada falla se documenta con la predicción escrita antes de conocer el resultado.

1. **Estado base** — Métricas de referencia sin falla.
2. **Predicción (antes)** — Qué se espera que ocurra y por qué.
3. **Activar falla** — Con una configuración especifica.
4. **Observar y medir** — Métricas, logs y/o trazas.
5. **Explicar** — Comparar predicción vs. realidad y analizar diferencias.
6. **Corregir y repetir** — Ajustar configuración y reejecutar para validar el efecto.

5. Observabilidad

Descubrir qué señales permiten entender la falla.

Métricas
p50/p95/p99, throughput, errores, CPU, memoria, GC, colas, circuit breakers.

Logs
Reconstruir qué ocurrió y cuándo.

Trazas
Seguir una solicitud a través de componentes y dependencias.

Y responder:
- ¿Qué ocurrió?
- ¿Dónde está la falla?
- ¿Cuáles no lo fueron?
- ¿Qué la está causando?
- ¿Qué señales fueron útiles?

6. Rúbrica de evaluación (100 puntos)

Criterio	Peso
1. Fallas configurables	20%
2. Diseño experimental y predicciones	20%
3. Ejecución, reproducibilidad y reversibilidad	15%
4. Observabilidad y evidencia	25%
5. Análisis de resultados y aprendizaje	10%
6. Corrección, reexperimentación y comunicación	10%
Total	**100%**

7. Entregables

Repositorio del código
Configuración de fallas, scripts, instrucciones.

Evidencia organizada
baseline/, experiments/, predictions/, results/, logs/, metrics/, traces/.

Informe técnico (máx. 10 páginas)
Tabla de experimentos, resultados, análisis y aprendizajes.

8. Tabla de experimentos (obligatoria)

ID	Familia	Falla	Configuración	Predicción	Resultado	Evidencia	Aprendizaje
A1	Latencia	Tributaria lenta	+800 ms, 30%	p95 +800 ms	p99 = 12 s	métricas + logs	timeout impedia abrir breaker
B1	BD	Pool agotado	max 10 conn, 5 min	aumento de errores	503 y timeouts	métricas + trazas	ajustar pool y retry

9. Condiciones de aprobación

- Entre 4 y 8 experimentos (1–2 por familia).
- Fallas configurables y sin redespliegue.
- Kill switch funcional.
- Predicción previa documentada.
- Evidencia suficiente por experimento.
- Procedimiento reproducible.
- Análisis y aprendizaje de cada experimento.

10. Lo que NO se debe premiar

- Solo "la aplicación se cayó".
- Un throw Exception como falla.
- Gráficas sin hipótesis.
- Solo promedios de latencia.
- Muchos dashboards sin interpretación.
- Una predicción siempre acertada (su valor está en el aprendizaje).

El corazón del proyecto

Fallas configurables · Experimentos · Observabilidad · Aprendizaje



No buscamos sistemas que nunca fallen.
Buscamos arquitectos capaces de entender qué ocurre cuando fallan.

UNIVERSIDAD EAFIT

ENTREGABLE 3

El corazón del proyecto

Fallas configurables, experimentación y observabilidad

Aplicaciones y Sistemas Escalables

Entender. Crear. Transformar.

1. Objetivo

Construir un laboratorio de fallas configurable y reproducible sobre el sistema desarrollado, para entender qué ocurre cuando falla, cómo detectarlo con observabilidad y qué decisiones de diseño tomar.

2. Fallas configurables

Cada falla debe cumplir las cinco condiciones:

1. **Sin redespliegue** — Se activa por archivo, variable o endpoint admin.
2. **Parametrizada** — Intensidad, probabilidad, alcance, duración.
3. **Focalizable** — Endpoint, componente o % de tráfico.
4. **Reversible en segundos** — Kill switch durante la demostración.
5. **Reproducible** — Misma config. + misma semilla = mismo comportamiento.

3. Cuatro familias de fallas (mínimo dos por familia)

A · Latencia inducida
- Latencia fija en dependencia
- Jitter (distribución lognormal)
- Solo en parte del tráfico
- Timeout controlado

B · Falla de base de datos
- Caída temporal de PostgreSQL
- Pool de conexiones agotado
- Bloqueo sobre un recurso
- Failover o disco lleno simulado

C · Degradación de tiempos
- Saturación de CPU
- Presión de memoria y GC
- Inanición del pool de hilos
- Consumidor pausado / lag

D · Riesgo de disponibilidad
- Caída de una instancia
- Partición de red servicio ↔ base
- DNS que no resuelve
- Respuestas 5xx externas
- Desfase de reloj

4. Método experimental

Cada falla se documenta con la predicción escrita antes de conocer el resultado.

1. **Estado base** — Métricas de referencia sin falla.
2. **Predicción (antes)** — Qué se espera que ocurra y por qué.
3. **Activar falla** — Con una configuración especifica.
4. **Observar y medir** — Métricas, logs y/o trazas.
5. **Explicar** — Comparar predicción vs. realidad y analizar diferencias.
6. **Corregir y repetir** — Ajustar configuración y reejecutar para validar el efecto.

5. Observabilidad

Descubrir qué señales permiten entender la falla.

Métricas
p50/p95/p99, throughput, errores, CPU, memoria, GC, colas, circuit breakers.

Logs
Reconstruir qué ocurrió y cuándo.

Trazas
Seguir una solicitud a través de componentes y dependencias.

Y responder:
- ¿Qué ocurrió?
- ¿Dónde está la falla?
- ¿Cuáles no lo fueron?
- ¿Qué la está causando?
- ¿Qué señales fueron útiles?

6. Rúbrica de evaluación (100 puntos)

Criterio	Peso
1. Fallas configurables	20%
2. Diseño experimental y predicciones	20%
3. Ejecución, reproducibilidad y reversibilidad	15%
4. Observabilidad y evidencia	25%
5. Análisis de resultados y aprendizaje	10%
6. Corrección, reexperimentación y comunicación	10%
Total	**100%**

7. Entregables

Repositorio del código
Configuración de fallas, scripts, instrucciones.

Evidencia organizada
baseline/, experiments/, predictions/, results/, logs/, metrics/, traces/.

Informe técnico (máx. 10 páginas)
Tabla de experimentos, resultados, análisis y aprendizajes.

8. Tabla de experimentos (obligatoria)

ID	Familia	Falla	Configuración	Predicción	Resultado	Evidencia	Aprendizaje
A1	Latencia	Tributaria lenta	+800 ms, 30%	p95 +800 ms	p99 = 12 s	métricas + logs	timeout impedia abrir breaker
B1	BD	Pool agotado	max 10 conn, 5 min	aumento de errores	503 y timeouts	métricas + trazas	ajustar pool y retry

9. Condiciones de aprobación

- Entre 4 y 8 experimentos (1–2 por familia).
- Fallas configurables y sin redespliegue.
- Kill switch funcional.
- Predicción previa documentada.
- Evidencia suficiente por experimento.
- Procedimiento reproducible.
- Análisis y aprendizaje de cada experimento.

10. Lo que NO se debe premiar

- Solo "la aplicación se cayó".
- Un throw Exception como falla.
- Gráficas sin hipótesis.
- Solo promedios de latencia.
- Muchos dashboards sin interpretación.
- Una predicción siempre acertada (su valor está en el aprendizaje).

UNIVERSIDAD EAFIT | Inspira Crea Transforma

Patrones de diseño



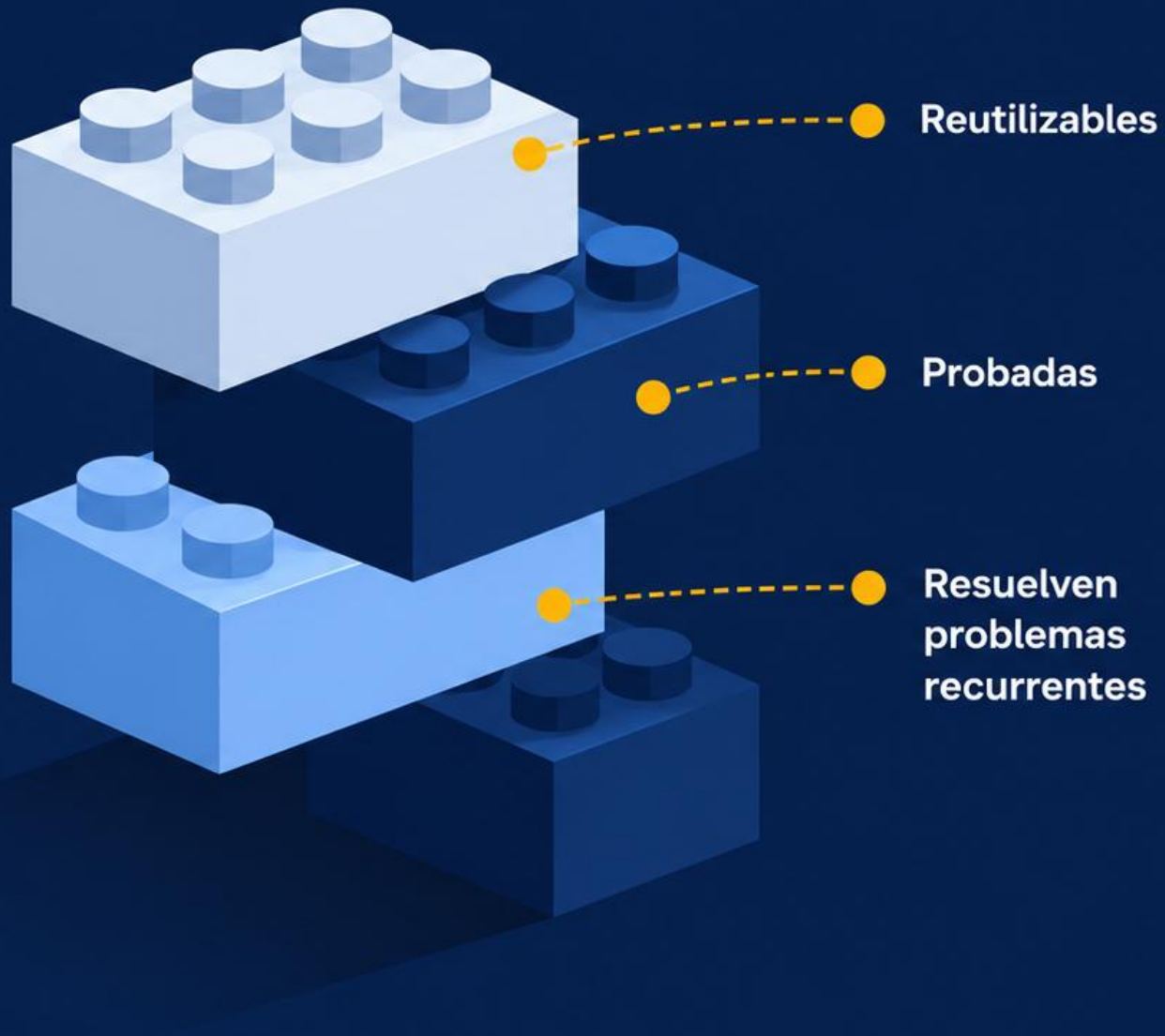


UNIVERSIDAD EAFIT | Inspira Crea Transforma

DEFINICIÓN

Patrones de diseño

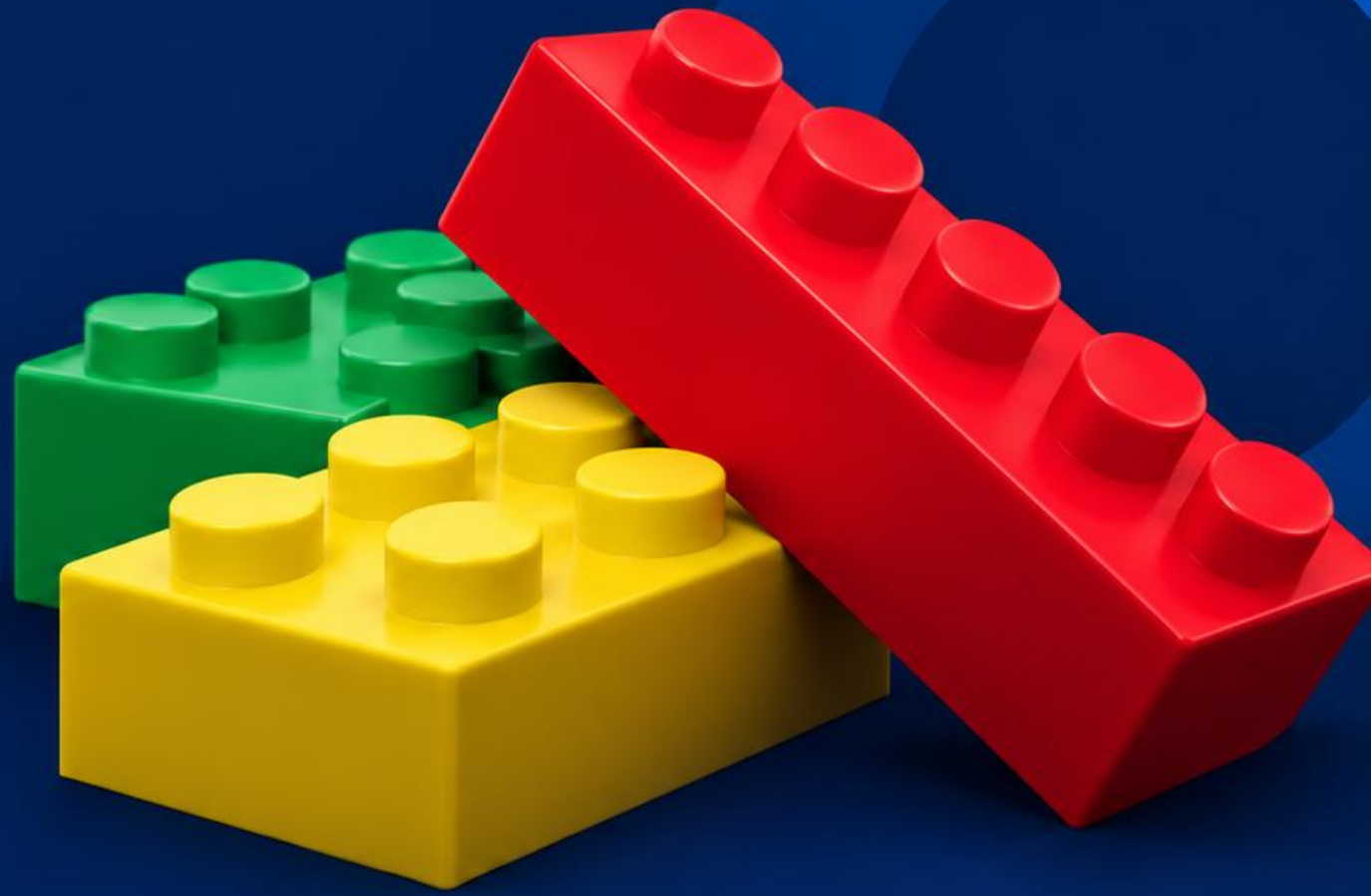
Soluciones probadas y reutilizables para estructurar componentes y resolver problemas recurrentes en el diseño de software.



UNIVERSIDAD EAFIT | Inspira Crea Transforma

Patrones de diseño

Soluciones probadas y reutilizables para estructurar componentes de software.



UNIVERSIDAD EAFIT | Inspira Crea Transforma

Funcionamiento del patrón Circuit Breaker

El interruptor de emergencia de las APIs

Protege tus sistemas.
Evita que una falla se convierta en un problema mayor.

Sistemas más resilientes para un mundo real

¿Qué es?

Es un patrón de diseño que evita fallas en cascada cuando un servicio externo está fallando o es lento.
Actúa como un interruptor entre tu aplicación y un servicio externo.
Si detecta fallas, "abre" el circuito y bloquea las peticiones por un tiempo.

Resultado: sistemas más resilientes, estables y con alta disponibilidad.

¿Cómo funciona? (en 5 pasos)

1. **Funcionamiento normal** — El circuito está cerrado y las peticiones pasan normalmente.
2. **Fallas repetidas** — Si ocurren fallas consecutivas (ej. 5), el circuito se abre.
3. **Circuito abierto** — Las peticiones se rechazan inmediatamente sin llamar al servicio.
4. **Tiempo de espera (Half-Open)** — Después de un tiempo (ej. 30 seg), se permite una petición de prueba.
5. **Recuperación** — Si la prueba es exitosa, el circuito se cierra. Si falla, vuelve a abrirse.

¿Por qué usarlo?

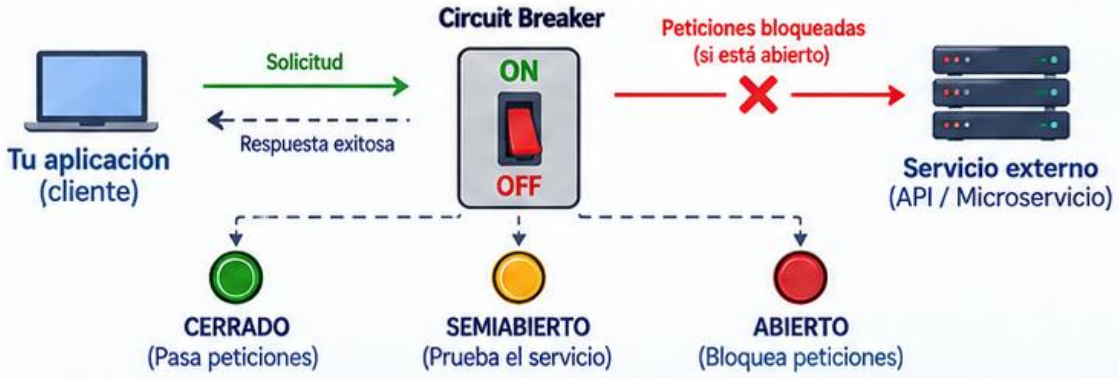
- Evita fallas en cascada
- Protege servicios dependientes
- Mejora la resiliencia del sistema
- Permite recuperación automática
- Aumenta la disponibilidad
- Mejor experiencia para el usuario

Actúa como un guardián entre tu aplicación y servicios inestables.

Estados del Circuit Breaker

- **Cerrado (Closed)** — Todo funciona bien. Las peticiones pasan.
- **Abierto (Open)** — Se detectan fallas. Las peticiones se bloquean.
- **Semiabierto (Half-Open)** — Se permite una petición de prueba para verificar si el servicio se recuperó.

Flujo de funcionamiento



Ejemplo práctico (llamada a un servicio de pagos)

1. **Todo bien** — El servicio responde y las transacciones se procesan. → CERRADO
2. **Fallas repetidas** — El servicio falla varias veces. Se alcanza el umbral de fallas. → ABIERTO
3. **Bloquea peticiones** — Las peticiones se rechazan sin llamar al servicio. → ABIERTO
4. **Prueba (Half-Open)** — Se permite una petición de prueba para verificar si el servicio responde. → SEMABIERTO
5. **Recuperación** — Si la prueba es exitosa, el circuito se cierra. → CERRADO

Buenas prácticas

- Define umbrales de fallas adecuados
- Configura tiempos de espera razonables
- Registra y monitorea los estados
- Implementa fallback (respuesta alternativa)
- Úsalo junto con timeouts y reintentos
- Ajusta la configuración por servicio



**Más control.
Más estabilidad.
Más confianza.**

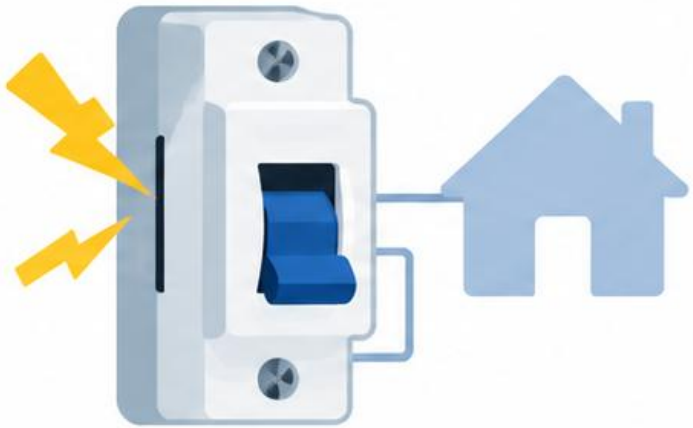
Circuit Breaker

Protege tu sistema cuando un servicio falla

UNIVERSIDAD EAFIT

Analógico cotidiano

El fusible de tu casa



Si hay un cortocircuito, el fusible se “abre” y corta la electricidad para evitar un incendio. No intentas encender la luz cada segundo; esperas a que se repare.

Netflix

Servicio de recomendaciones



Cuando el servicio de recomendaciones falla, el circuit breaker se abre y muestra un mensaje en lugar de colgar la app.

Bancos digitales

Transferencias interbancarias



Si el servicio de transferencias está caído, el circuit breaker activa un fallback y notifica al usuario sin saturar los servidores.

UNIVERSIDAD EAFIT | Inspira Crea Transforma

Patrón Try-Catch

Gestionar errores para un sistema más resiliente

Los errores son inevitables.
El patrón Try-Catch nos permite anticiparlos, manejarlos y mantener la estabilidad de nuestras aplicaciones.

¿Qué es?

El patrón Try-Catch es un mecanismo de control de excepciones que permite manejar errores en tiempo de ejecución, evitando que la aplicación se detenga y facilitando una respuesta controlada.

No elimina los errores, pero nos da una forma segura de gestionarlos.

Elementos principales

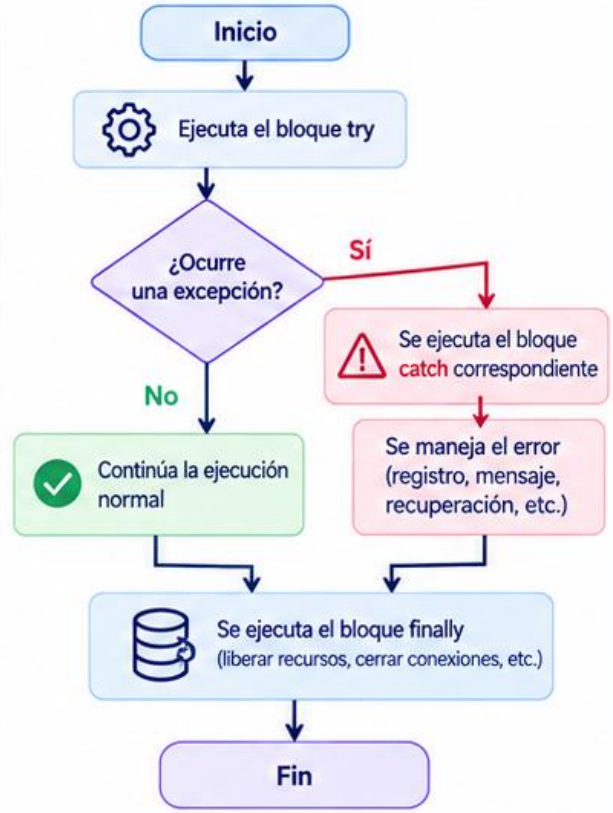
- **try** — Bloque donde se ejecuta el código que puede generar una excepción.
- **catch** — Bloque que captura y maneja la excepción.
- **finally** — Bloque opcional que siempre se ejecuta, haya o no una excepción (útil para liberar recursos).
- **throw** — Permite lanzar una excepción de forma explícita.

Ejemplo en un sistema real (Hospital Infantil)

Obtener información de un paciente desde un servicio externo.

```java
public Paciente obtenerPaciente(String id) {
  try {
    return servicioPacientes.buscarPorId(id);
  } catch (PacienteNoEncontradoException e) {
    logger.warn("Paciente no encontrado: " + id);
    return null;
  } catch (Exception e) {
    logger.error("Error consultando paciente", e);
    throw new ServicioNoDisponibleException(
      "El servicio de pacientes no está disponible");
  } finally {
    metricas.incrementarConsulta();
  }
}
```

¿Cómo funciona?



Buenas prácticas

- **Captura excepciones específicas** — Evita capturar Exception de forma genérica.
- **Registra la excepción** — Incluye contexto relevante para facilitar el diagnóstico.
- **Toma acciones adecuadas** — Define una estrategia: reintento, valor por defecto, mensaje al usuario, etc.
- **No ocultes errores** — Evita bloques catch vacíos.
- **Usa finally para liberar recursos** — Cerrar conexiones, archivos, hilos, etc.

Beneficios

- Mayor estabilidad
- Mejor experiencia de usuario
- Permite recuperación controlada
- Facilita el diagnóstico y monitoreo

> "Un buen manejo de errores no evita que fallen las cosas, pero sí evita que todo falle."



UNIVERSIDAD EAFIT

Patrón de Bulkhead

Aisla recursos para que un fallo no afecte todo el sistema

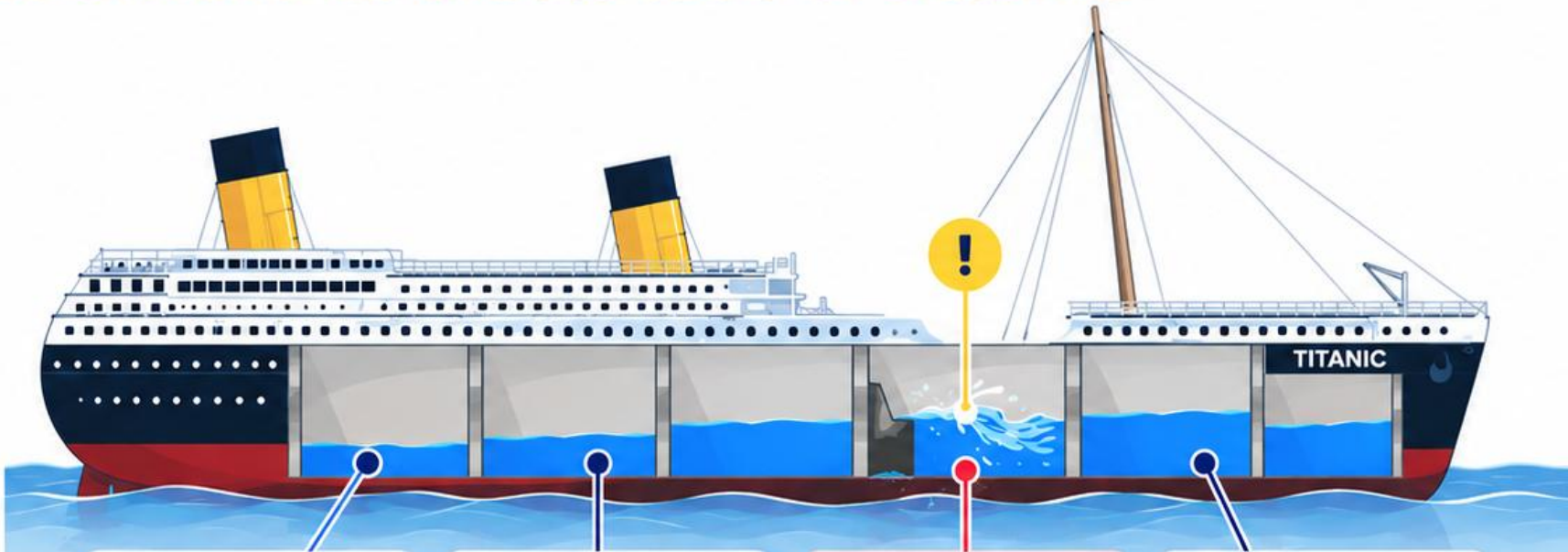
Definición

El patrón Bulkhead aisla recursos (hilos, conexiones, memoria, pools) para que un fallo en una parte del sistema no afecte al resto, exactamente como los mamparos estancos de un barco que evitan que se hunda todo si entra agua en un compartimento.

Cómo funciona

Se crean "compartimentos" independientes de recursos:

1. Pool de hilos separado por cliente o servicio.
2. Límite de conexiones o cuota de memoria por "bulkhead".
3. Si un bulkhead se satura, solo ese compartimento falla; los demás siguen operando.



En software

Kubernetes y contenedores

Cada microservicio tiene límites de CPU y memoria por pod (requests/limits). Si un servicio de pagos consume toda la memoria, el servicio de catálogo de productos sigue respondiendo normalmente.

Amazon AWS
(Service Quotas y Bulkhead en Lambda)

Un cliente que hace millones de peticiones a la API de pagos tiene su propio pool de conexiones; si se agota, solo ese cliente es afectado, no todo el marketplace.

Patrón Retry (Reintento)

Reintenta automáticamente operaciones fallidas

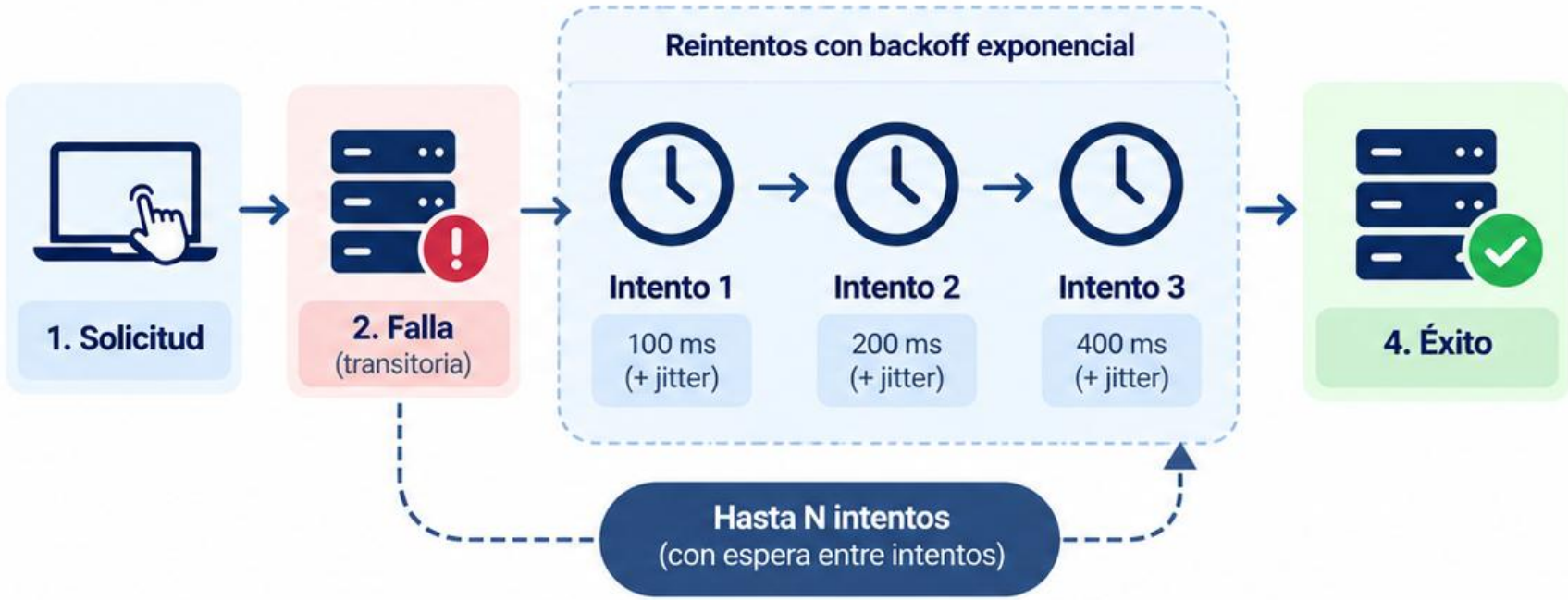
UNIVERSIDAD EAFIT

Definición

El patrón Retry reintenta automáticamente una operación fallida que se considera transitoria (temporal), mejorando la probabilidad de éxito sin intervención manual.

Cómo funciona

1. Se define un número máximo de intentos (ej. 3-5).
2. Se aplica backoff exponencial (espera 100 ms, luego 200 ms, 400 ms...) más jitter (aleatoriedad) para evitar la "tormenta de peticiones" (*thundering herd*).
3. Se combinan con circuit breaker para no reintentar indefinidamente.



Estratégico

Usar backoff exponencial + jitter para evitar la "tormenta de peticiones".

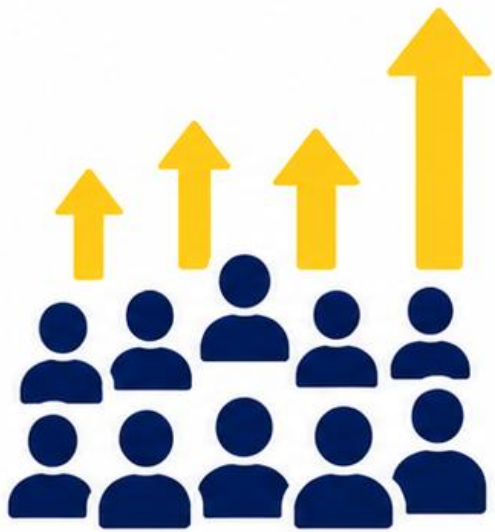
Complemento

Combinar con circuit breaker para no reintentar indefinidamente.

UNIVERSIDAD EAFIT

Pruebas de estrés

¿Cómo se comporta el sistema bajo alta carga?



Más usuarios y más carga





Identificar límites y mejorar la resiliencia

UNIVERSIDAD EAFIT

Pruebas de estrés

Ejemplo: Validador de envíos SAP



UNIVERSIDAD EAFIT

Pruebas de estrés

Validan la capacidad del sistema bajo alta carga

Definición

Su propósito es evaluar de manera rigurosa el **comportamiento no funcional** del sistema bajo distintos niveles de carga, identificar **cuellos de botella, puntos de quiebre** y verificar el cumplimiento de los **acuerdos de nivel de servicio (SLA)** definidos para el entorno de producción. Este ejercicio de pruebas responde a la necesidad de validar la capacidad del sistema ante **escenarios de uso proyectados**, garantizando la **continuidad operativa** del negocio y la **experiencia del usuario final** ante condiciones de carga variable.



https://xxxxxxx.yyyyyy-api.us-east-1.amazonaws.com/lab/validadorsap/envios-sap

Alcance de las Pruebas

- Prerrequisitos para la ejecución
  - El ambiente debe encontrarse disponible, estable y sin cambios en curso.
  - Se espera disponibilidad de ventana mínima de 3 horas para las pruebas planteadas.
  - Los datos de prueba deben estar preparados y contar con variabilidad.
  - Flujo validado: peticiones con 2 recibos por solicitud simulando escenarios reales.
  - Herramienta de pruebas: Apache JMeter 5.6.3, con scripts previamente validados mediante pruebas de sanidad.
  - Disponibilidad de dashboards de monitoreo en herramienta de observabilidad para el seguimiento de métricas del servidor durante la ejecución, con acompañamiento requerido por parte del equipo cliente.
  - Se deberá contar con una ventana de ejecución donde se garantice que el orquestador estará deshabilitado para no saturar el API Gateway.

Criterios de aceptación para la prueba

1. Volumen estimado
   - 3.000 peticiones por día.
   - 375 peticiones por hora, considerando una disponibilidad de 8 horas diarias del servidor.
   - Flujo validado: peticiones con 2 recibos por solicitud simulando escenarios reales.

2. Consideraciones del ambiente el ambiente QA equivale al 80% de la capacidad de producción, base para las proyecciones.
   - Se espera disponibilidad de ventana mínima de 3 horas para las pruebas planteadas.
   - Acompañamiento requerido por parte del cliente para el monitoreo del servidor desde la herramienta de observabilidad.
   - El uso de recursos estará limitado a CPU y memoria ≤ 80% como umbral máximo esperado.

3. Tiempos de respuesta (SLA/SLO)
   - Tiempo de respuesta esperado:
     - ≤ 15.000 ms en condiciones estándar.
     - ≤ 20.000 ms para condiciones de alta demanda (picos).

UNIVERSIDAD EAFIT

Alcance de la prueba

- **Prueba de carga**
  - Se enviarán peticiones con 2 recibos por solicitud, basadas en un volumen de 3.000 peticiones diarias.
  - Duración de la prueba: 1 hora.
  - Porcentaje de error máximo permitido: 5% para errores de tipo HTTP ≥ 500.
- **Prueba de estrés**
  - Se enviarán peticiones con 2 recibos por solicitud. o Duración de la prueba: 1 hora y 20 minutos.
  - Ejecución mediante esquema escalonado, incrementando progresivamente la carga con un aumento de 3 usuarios por escalón, por encima del límite base hasta alcanzar el punto de quiebre, definido como:
    - Más del 50% de errores HTTP ≥ 500

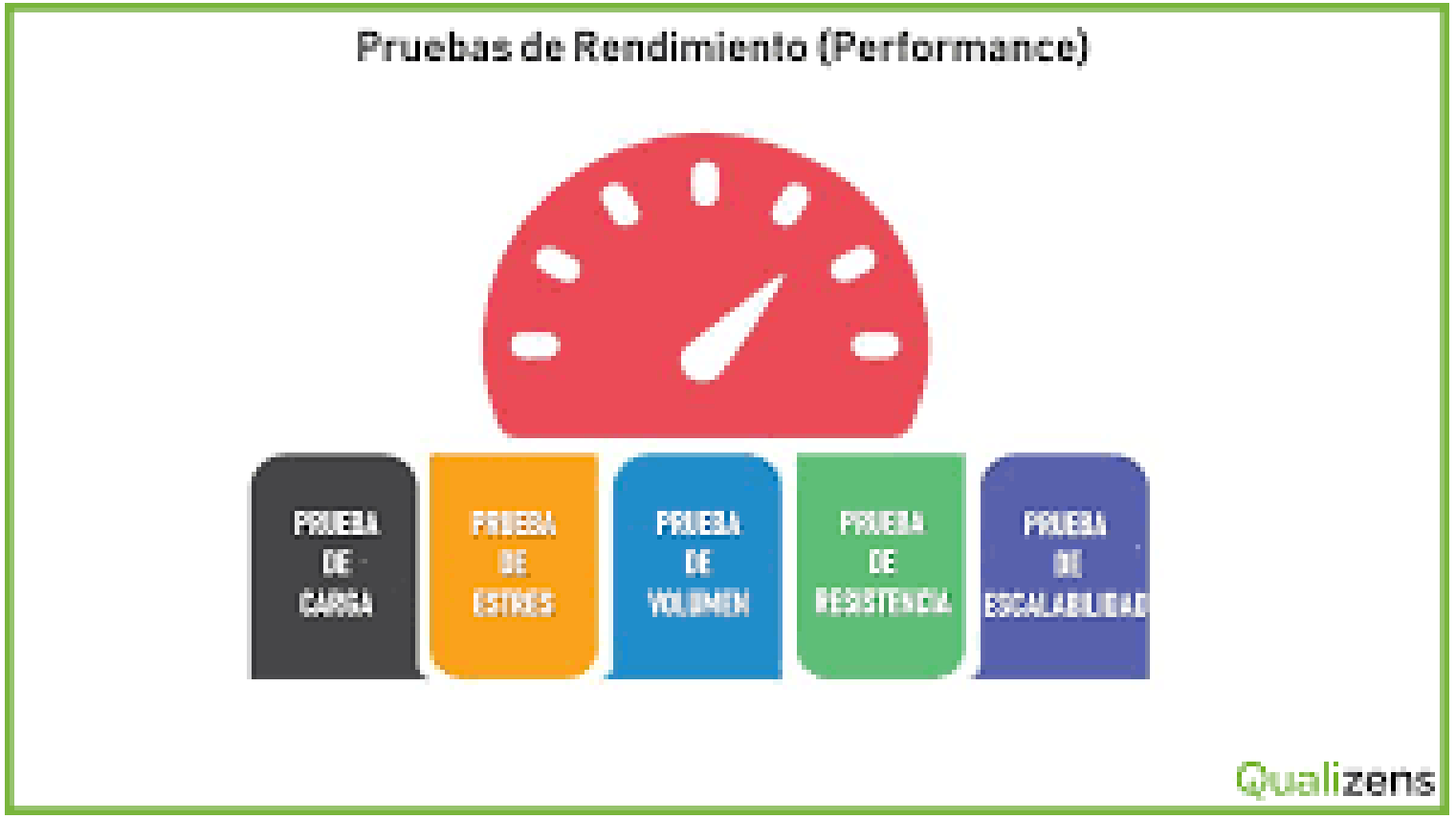
UNIVERSIDAD EAFIT

Objetivos técnicos específicos

Las pruebas realizadas tienen como objetivos técnicos específicos:

- Rendimiento bajo carga: Verificar que los tiempos de respuesta se mantengan dentro de los umbrales de SLA para los percentiles P50, P95 y P99 bajo la carga nominal esperada.
- Escalabilidad: Determinar si el sistema es capaz de manejar incrementos de carga de manera lineal o si existen cuellos de botella que limiten la escalabilidad horizontal.
- Estabilidad y durabilidad: Garantizar que el sistema no experimente degradación progresiva durante períodos sostenidos de operación bajo carga.
- Identificación de punto de quiebre: Establecer el umbral máximo de carga antes del cual el sistema entra en estado de saturación irrecuperable.
- Cumplimiento de SLA: Contrastar los resultados obtenidos con los acuerdos contractuales y operativos definidos para el entorno productivo.

Resultados



UNIVERSIDAD EAFIT

Estado general del sistema

2.1 Estado General del Sistema

El sistema presenta un comportamiento ESTABLE bajo carga nominal con oportunidades de mejora identificadas en escenarios de alta concurrencia. Se recomienda atención inmediata a los cuellos de botella identificados antes del pase a producción.

2.2 Principales Riesgos Identificados

ID	Riesgo	Severidad	Área Afectada
R-01	Degradación de tiempos de respuesta bajo alta concurrencia	Alto	INFRAESTRUCTURA
R-02	Saturación de pool de conexiones a base de datos	Bajo	BD
R-03	Ausencia de mecanismos de circuit breaker en servicios críticos	Medio	API
R-04	Limite de conexiones por rate limit	Alto	API

UNIVERSIDAD EAFIT

Cumplimiento de SLA

2.3 Cumplimiento de SLA

A continuación, se resume el cumplimiento de los indicadores clave de rendimiento respecto a los SLA definidos:

Prueba de Carga:

Indicador	Umbral SLA	Resultado Obtenido	Estado
Percentil 95 (P95)	≤ 15000 ms	12909 ms	✓
Percentil 99 (P99)	≤ 20000 ms	16487 ms	✓
Tasa de Error	< 5 %	0 %	✓
Uso de CPU (Pico)	< 80 %	pendiente %	pendiente
Uso de Memoria (Pico)	< 80 %	pendiente %	pendiente

Prueba de Estrés:

Indicador	Umbral SLA	Resultado Obtenido	Estado
Percentil 95 (P95)	≤ 15000 ms	19726 ms	✗
Usuarios concurrentes	80	75	✗
Percentil 99 (P99)	≤ 20000 ms	23909 ms	✗
Tasa de Error	< 5 %	94.51 %	✗

UNIVERSIDAD EAFIT

Conclusión

ℹ Recomendación Ejecutiva

Se recomienda **implementar optimizaciones en el manejo de concurrencia, control de rate limiting y tiempos de respuesta del servicio, así como ajustar la capacidad del sistema para soportar cargas superiores a 75 usuarios concurrentes**, como condición previa al despliegue en producción. Adicionalmente, se sugiere realizar una nueva ronda de pruebas de estrés posterior a las mejoras implementadas, con el fin de validar la estabilidad del sistema.

Se deberá realizar seguimiento continuo de métricas clave (tiempos de respuesta, tasa de errores, throughput y uso de recursos) durante las primeras **2 a 4 semanas post-despliegue**, con el objetivo de identificar de manera temprana posibles degradaciones en escenarios reales de operación y garantizar el cumplimiento sostenido de los SLA definidos.

Criterios de éxito y definición de SLA

3.1 Tiempos de Respuesta

Percentil	Umbral Máximo Permitido	Justificación
P90	≤ 15000 ms	Indicador de comportamiento bajo carga moderada
P99	≤ 20000 ms	Límite para casos extremos sin impacto perceptible
Tiempo Máximo Absoluto	≤ 20000 ms	Umbral de timeout configurado en cliente/gateway

3.2 Tasa de Error

La tasa de error total (incluyendo errores HTTP 4xx y 5xx, timeouts de conexión y fallos de validación funcional) no deberá superar el 5% durante ninguna ventana de medición de 60 segundos bajo carga nominal.

Para escenarios de estrés, se admite una tasa de error incremental no mayor a 5 % en el punto de saturación identificado, siempre que el sistema sea capaz de recuperarse autónomamente en un tiempo máximo de 20 segundos.

3.3 Throughput (Capacidad de Procesamiento)

El sistema deberá ser capaz de procesar al menos 6.3 TPS transacciones por segundo de manera sostenida durante la ventana de carga nominal de 1 hora. Este valor representa el 80% de la capacidad máxima proyectada para el entorno productivo.

3.4 Indicadores de Infraestructura

Recurso	Umbral Máximo (Carga Normal)	Umbral Máximo (Estrés)	Unidad
CPU — Servidores de Aplicación	≤ 3%	≤ 5%	% promedio en ventana de 5 min
Memoria RAM — Aplicación	≤ 19%	≤ 19%	% del total disponible

UNIVERSIDAD EAFIT

Resultados detallados x escenario

4.1.2 Métricas Obtenidas

Métrica	Valor Obtenido	Umbral SLA	Estado
Tiempo Promedio de Respuesta	9616 ms	—	—
Tiempo Mínimo de Respuesta	6195 ms	—	—
Tiempo Máximo de Respuesta	17594 ms	—	—
Percentil 90 (P90)	10726 ms	15000 ms	✓
Percentil 95 (P95)	12909 ms	—	—
Percentil 99 (P99)	16487 ms	≤ 20000 ms	✓
Throughput (TPS)	0.10 s	—	—
Total de Solicitudes Enviadas	375	375	✓
Total de Errores	0	—	—
Tasa de Error (%)	0 %	5 %	✓

UNIVERSIDAD EAFIT

Resultados obtenidos

4.1.3 Evidencias Gráficas

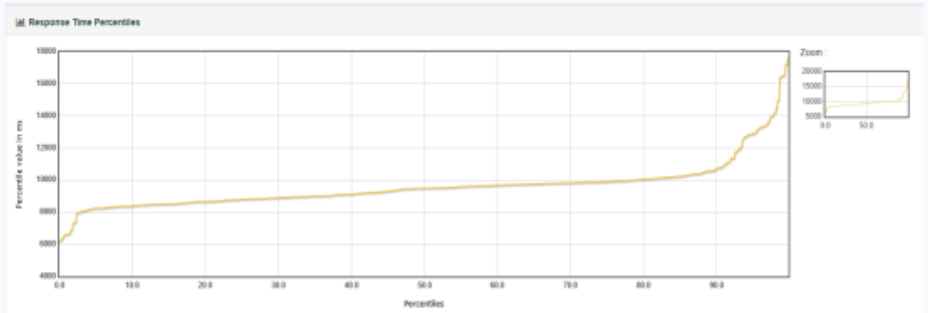


Figura 4.1.1 — Gráfica de Tiempos de Respuesta durante la Prueba de Carga (eje X: tiempo, eje Y: latencia en ms). Incluir curvas P50, P95, P99.

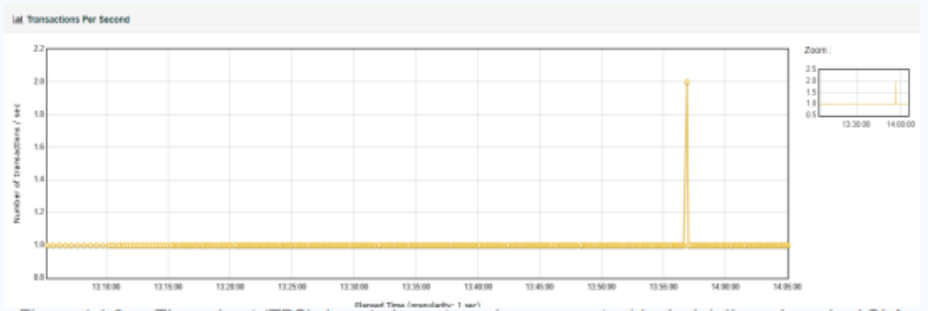


Figura 4.1.2 — Throughput (TPS) durante la ventana de carga sostenida. Incluir línea de umbral SLA.



Figura 4.1.3 — Métricas de infraestructura: CPU y Memoria del servidor de aplicación durante la prueba.

UNIVERSIDAD EAFIT

Pruebas de stress

4.2.2 Métricas Detalladas por Nivel de Carga

Nivel de Carga (VU)	TPS	Tiempo Promedio (ms)	P95 (ms)	P99 (ms)	Error Rate Req/s
3 VU — Base	3	13276 ms	10556 ms	10556 ms	HTTP 429: 0 req/s HTTP 500: 0 req/s HTTP 504: 0 req/s
21 VU — Nivel 2	14	4498 ms	10330 ms	10330 ms	HTTP 429: 0 req/s HTTP 500: 1 req/s HTTP 504: 0 req/s
40 VU — Nivel 3	13	247 ms	7717 ms	7717 ms	HTTP 429: 0 req/s HTTP 500: 0 req/s HTTP 504: 0 req/s
70 VU — Nivel 4	13	298 ms	28985 ms	28985 ms	HTTP 429: 0 req/s HTTP 500: 1 req/s HTTP 504: 1 req/s
75 VU — Punto de quiebre	20	887 ms	29234 ms	29234 ms	HTTP 429: 300 req/s HTTP 500: 1 req/s HTTP 504: 1 req/s

UNIVERSIDAD EAFIT

Pruebas de stress

4.2.3 Identificación del Punto de Quiebre y Degradación

El sistema comenzó a mostrar señales de degradación significativa a partir de aproximadamente 70 usuarios concurrentes. Los síntomas observados incluyeron:

- Incremento abrupto del P99 hasta valores cercanos a 28.985 ms, superando ampliamente los niveles observados en cargas previas y evidenciando alta latencia en un porcentaje de las transacciones.
- Aparición de errores tipo HTTP 500 (errores internos) y HTTP 504 (timeouts de gateway), con una frecuencia aproximada de 1 req/s para cada tipo de error, indicando fallas en el procesamiento y tiempos de respuesta excedidos.
- Alta variabilidad en los tiempos de respuesta (spikes), reflejada en la diferencia entre el tiempo promedio y los percentiles altos, lo que evidencia inestabilidad bajo condiciones de alta concurrencia.

El punto de quiebre definitivo se identificó en 75 usuarios concurrentes, momento en el cual se activaron mecanismos de limitación de tráfico (HTTP 429 - rate limiting) con una tasa aproximada de 300 req/s, acompañado de errores 500 y 504. Esta condición provocó una degradación severa del servicio, haciendo inviable la continuidad operativa bajo ese nivel de carga.

Posterior a la saturación, el sistema no demostró una capacidad de recuperación autónoma dentro del rango de la prueba, ya que el rate limiting impidió la normalización del flujo de solicitudes. El comportamiento observado sugiere la necesidad de intervención a nivel de configuración o escalamiento, especialmente en los mecanismos de control de tráfico y capacidad del backend.

Durante el escenario de estrés, el sistema presentó un incremento moderado en el uso de CPU, alcanzando picos de hasta 4.4%, sin evidenciar saturación. El consumo de memoria se mantuvo estable alrededor del 18.8%, descartando posibles fugas de memoria.

Sin embargo, se observó un incremento significativo en el tráfico de red, con picos de hasta 1020 Bytes/s en transmisión, lo que indica que la carga del sistema impacta principalmente los componentes de comunicación y transferencia de datos.

En este contexto, el sistema no presenta cuellos de botella a nivel de procesamiento o memoria, pero podría estar limitado por dependencias externas o I/O de red.

Pruebas de stress

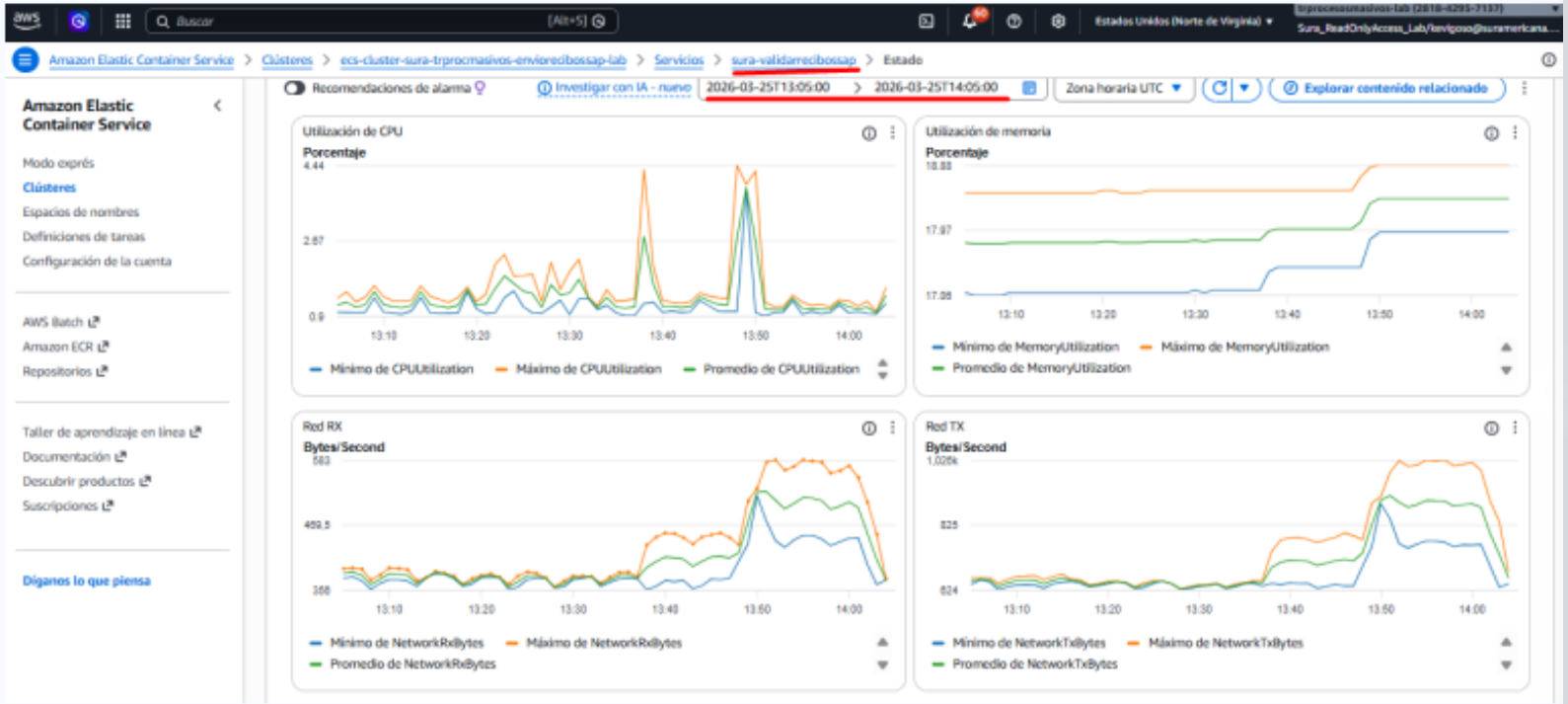


Figura 4.2.3 — Saturación de recursos de infraestructura en el punto de quiebre (CPU, Memoria, Conexiones BD).

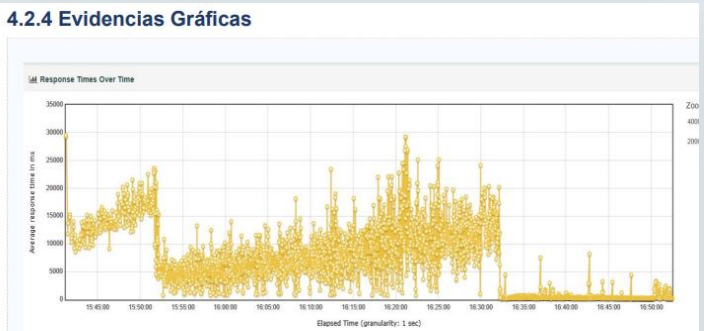


Figura 4.2.1 — Curva de escalamiento de tiempos de respuesta. Marcar claramente el quiebre identificado.



olución de la tasa de errores durante el esca

UNIVERSIDAD EAFIT

Análisis de cuellos de botella

5.1 Procesamiento del Endpoint de Recibos.

5.1.1 Descripción del Problema

Durante la ejecución de los escenarios de carga y estrés, se identificó una degradación significativa en el rendimiento del endpoint:
https://xxxxxxxx.yyyyyy-api.us-east-1.amazonaws.com/lab/validadorsap/envios-sap
correspondiente a la consulta y validación de recibos hacia sistemas externos (SAP). Los tiempos de respuesta observados, especialmente en percentiles altos (P95 y P99), evidencian que este endpoint concentra una porción crítica del tiempo total de la transacción, alcanzando valores superiores a 29.000 ms en condiciones de alta concurrencia.

UNIVERSIDAD EAFIT

Análisis de recursos asociados

Durante la ejecución de las pruebas, se realizó el monitoreo de los principales recursos del sistema (CPU, memoria, red y comportamiento de I/O), con los siguientes hallazgos:

- **Uso de Memoria** El consumo de memoria se mantuvo estable durante los escenarios de carga y estrés, con valores promedio cercanos al 18% y sin variaciones abruptas ni crecimiento progresivo.
- **Interpretación**: No se evidencian fugas de memoria ni presión sobre este recurso, por lo que se descarta como causa de la degradación observada en los tiempos de respuesta.
- **Uso de CPU** La utilización de CPU presentó valores bajos, con picos máximos cercanos al 4.4% durante el escenario de estrés.
- **Interpretación**: El bajo uso de CPU indica que el procesamiento interno del servicio no es un factor limitante, y que la aplicación no está siendo exigida a nivel de cómputo.
- **Comportamiento de Red** Se evidenció un incremento significativo en el tráfico de red durante el escenario de estrés, especialmente en la transmisión (TX), alcanzando valores cercanos a 1020 Bytes/s
- **Interpretación**: Este comportamiento sugiere que el endpoint depende fuertemente de integraciones externas (SAP), donde la latencia y tiempos de respuesta de dichos servicios impactan directamente el rendimiento global.
- **Uso de Disco / I/O** No se observaron métricas que evidencien saturación o uso elevado de operaciones de disco durante la ejecución de las pruebas.
- **Interpretación**: El sistema no presenta cuellos de botella asociados a operaciones de lectura/escritura en disco.

UNIVERSIDAD EAFIT

Analisis de causa raíz

Las causas técnicas más relevantes incluyen:

- Dependencia de servicios externos (SAP) con latencias no controladas, lo que introduce variabilidad en los tiempos de respuesta.
- Procesamiento síncrono sin mecanismos de desacoplamiento (asincronía o colas), afectando la escalabilidad.
- Ausencia de estrategias de caché para consultas repetitivas.
- Falta de optimización para escenarios concurrentes, evidenciada en el incremento abrupto de los percentiles altos

Capa	Problema Identificado	Impacto Técnico
Lógica de negocio	Procesamiento síncrono de validaciones contra sistemas externos (SAP)	Incremento en latencia total
Integración externa	Dependencia de servicios externos con tiempos de respuesta variables	Alta variabilidad en percentiles (P95/P99)
Serialización / Payload	Posible manejo de respuestas sin optimización o compresión	Aumento en tiempos de transferencia
Red / Gateway	Paso por API Gateway con posibles políticas de control (throttling / rate limit)	Introducción de latencias adicionales

UNIVERSIDAD EAFIT

Impacto en rendimiento y negocio

Este cuello de botella impacta directamente la capacidad de escalabilidad del sistema, evidenciando un comportamiento de degradación no lineal bajo condiciones de alta concurrencia. A partir de aproximadamente 70 usuarios concurrentes, los tiempos en percentiles altos superan ampliamente los valores aceptables, exponiendo a un porcentaje de usuarios a latencias críticas (> 20 segundos).

El impacto en negocio se traduce en:

- Retrasos en el procesamiento de transacciones
- Deterioro de la experiencia del usuario
- Riesgo de abandono de operaciones
- Posible afectación en integraciones críticas con sistemas externos

✕ **Cuello de Botella Crítico**

Este problema tiene prioridad **ALTA** y debe ser abordado antes de un despliegue a producción ya que compromete directamente el cumplimiento de SLA y la experiencia del usuario final.

Saturación

Comportamiento Bajo Alta Concurrencia

- Bajo condiciones de alta concurrencia (> 70 usuarios concurrentes), el sistema exhibió un patrón claro de saturación, caracterizado por:
  - Incremento abrupto en los tiempos de respuesta (P99 ~ 29.000 ms)
  - Aparición de errores HTTP 500 y 504
  - Activación masiva de mecanismos de rate limiting (HTTP 429)

5.2.2 Límites del Sistema Identificados

Recurso / Componente	Comportamiento Observado	Conclusión
API Gateway	Activación de rate limiting (429 ~ 300 req/s)	Límite de capacidad alcanzado
Backend / Servicio	Errores 500 bajo carga	Fallos en procesamiento interno
Integración externa (SAP)	Incremento de latencia y timeouts (504)	Dependencia crítica no resiliente

UNIVERSIDAD EAFIT

Tipos de errores generados



5.2.3 Tipos de Errores Generados

Los errores observados responden a un patrón típico de **degradación en cascada**:

1. **Incremento de latencia en servicios dependientes**, elevando los tiempos de respuesta globales.
2. **Timeouts (HTTP 504)** debido a tiempos de espera excedidos en la cadena de procesamiento.
3. **Errores internos (HTTP 500)** generados por fallos en la lógica bajo condiciones de estrés.
4. **Activación de rate limiting (HTTP 429)** como mecanismo de protección ante sobrecarga, bloqueando masivamente nuevas solicitudes.

Este comportamiento evidencia la ausencia o insuficiencia de mecanismos de resiliencia como:

- Circuit breakers
- Retry policies controladas
- Backpressure

5.2.4 Impacto en Usuarios Finales

Durante el punto de saturación (**75 usuarios concurrentes**), se estima que **la mayoría de las solicitudes fueron rechazadas o fallaron**, con una tasa de error cercana al **94%** debido principalmente al rate limiting.

El impacto en usuarios finales incluye:

- **Interrupción total del servicio**
- **Fallos visibles en transacciones**
- **Imposibilidad de completar operaciones críticas**
- **Alta probabilidad de abandono de usuarios**

En un entorno productivo, este comportamiento podría traducirse en:

- Pérdida significativa de transacciones
- Incremento en tickets de soporte
- Afectación directa en la percepción del servicio

UNIVERSIDAD EAFIT

Recomendaciones técnicas y estratégicas

- Validar con el negocio si el consumo concurrente cercano o superior a 70–75 usuarios corresponde a un escenario real o esperado.
- Identificar los flujos críticos de mayor impacto (transacciones con integración a SAP) y priorizarlos dentro de las pruebas.
- Analizar patrones reales de tráfico (horas pico, volumen de solicitudes, concurrencia máxima esperada).
- Ajustar los modelos de carga para representar con mayor precisión la realidad operativa, incluyendo:
  - Distribución de usuarios concurrentes
  - Frecuencia de invocación del endpoint
  - Comportamiento de picos de tráfico

- Optimización de Rate Limiting y Control de Tráfico
- Dado que el principal punto de falla bajo estrés fue la activación masiva de HTTP 429 (rate limiting), se recomienda:
- Revisar y ajustar las políticas de throttling configuradas en el gateway para evitar bloqueos abruptos del servicio.
- Implementar estrategias de control de tráfico más inteligentes, como:
  - Limitación progresiva en lugar de cortes abruptos
  - Priorización de solicitudes críticas
  - Uso de colas para desacoplar la carga

6.3.2 Configuración de Timeouts

Se recomienda estandarizar y alinear los timeouts en todos los niveles del sistema para evitar acumulación de solicitudes y consumo innecesario de recursos:

Nivel	Acción recomendada
Cliente	Manejo de errores controlado y feedback al usuario
API Gateway	Ajustar timeout para evitar respuestas tardías (504)
Backend	Cancelación temprana de procesos lentos
Integraciones externas	Definir timeouts estrictos y controlados

UNIVERSIDAD EAFIT

Patrones de resiliencia

Para mitigar la degradación observada bajo estrés, se recomienda implementar:

• Circuit Breaker Evitar propagación de fallos cuando servicios externos (SAP) presenten alta latencia.

• Retry con Backoff Exponencial Aplicado únicamente a operaciones seguras, evitando sobrecargar aún más el sistema.

• Bulkhead Pattern Aislar recursos para evitar que operaciones lentas afecten todo el sistema.

• Caché (Redis o similar) Reducir llamadas repetitivas a servicios externos, especialmente en consultas frecuentes.

• Procesamiento Asíncrono

Desacoplar operaciones no críticas mediante colas (SQS, Kafka, RabbitMQ), reduciendo tiempos de respuesta y presión sobre el sistema.

UNIVERSIDAD EAFIT